Homework 5 Exponential smoothing

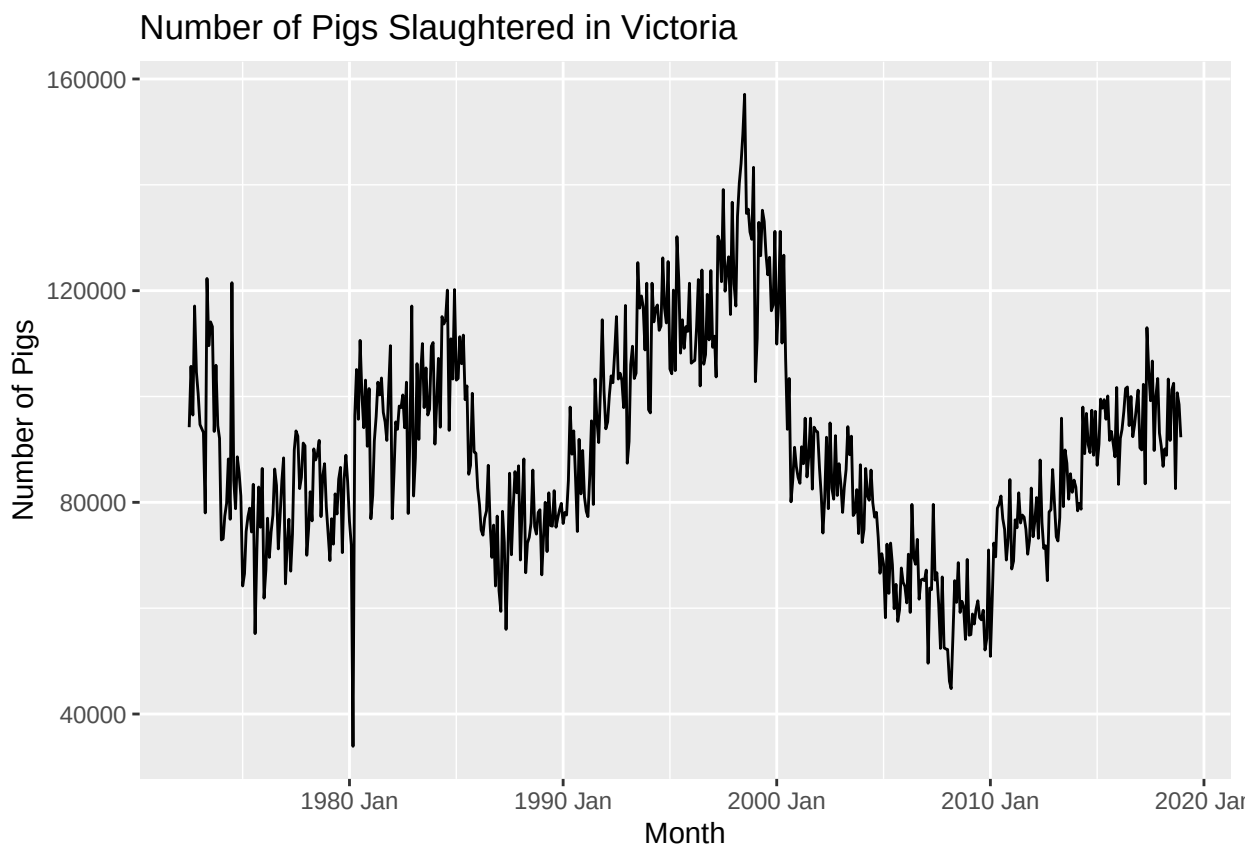
Mehreen Ali Gillani

2026-03-04

8. **1 Consider the the number of pigs slaughtered in Victoria, available in the aus_livestock dataset.**

a. Use the ETS() function to estimate the equivalent model for simple exponential smoothing. Find the optimal values of Alpha and l theta , and generate forecasts for the next four months.

```
# Load the aus_livestock dataset and filter for Victoria pigs
victoria_pigs <- aus_livestock %>%
  filter(State == "Victoria", Animal == "Pigs") %>%
  select(Month, Count)


# Plot the time series
victoria_pigs %>%
  autoplot(Count) +
  labs(title = "Number of Pigs Slaughtered in Victoria",
       y = "Number of Pigs",
       x = "Month")
```




```r
# Fit simple exponential smoothing model
# ETS(A,N,N) represents simple exponential smoothing with additive errors
fit <- victoria_pigs %>%
  model(ETS(Count ~ error("A") + trend("N") + season("N")))

# Display model information including optimal parameters
report(fit)
```

```
## Series: Count
## Model: ETS(A,N,N)
##   Smoothing parameters:
##     alpha = 0.3221247
##
##   Initial states:
##      l[0]
##  100646.6
##
##   sigma^2:  87480760
##
##      AIC     AICc      BIC
## 13737.10 13737.14 13750.07
```

```r
# Extract the optimal parameters
alpha <- fit %>%
  tidy() %>%
  filter(term == "alpha") %>%
  pull(estimate)
```

```r
l0 <- fit %>%
  tidy() %>%
  filter(term == "initial") %>%
  pull(estimate)

cat("Optimal alpha (smoothing parameter):", alpha, "\n")
```

```
## Optimal alpha (smoothing parameter): 0.3221247
```

```r
cat("Optimal initial level (l0):", l0, "\n")
```

```
## Optimal initial level (l0):
```

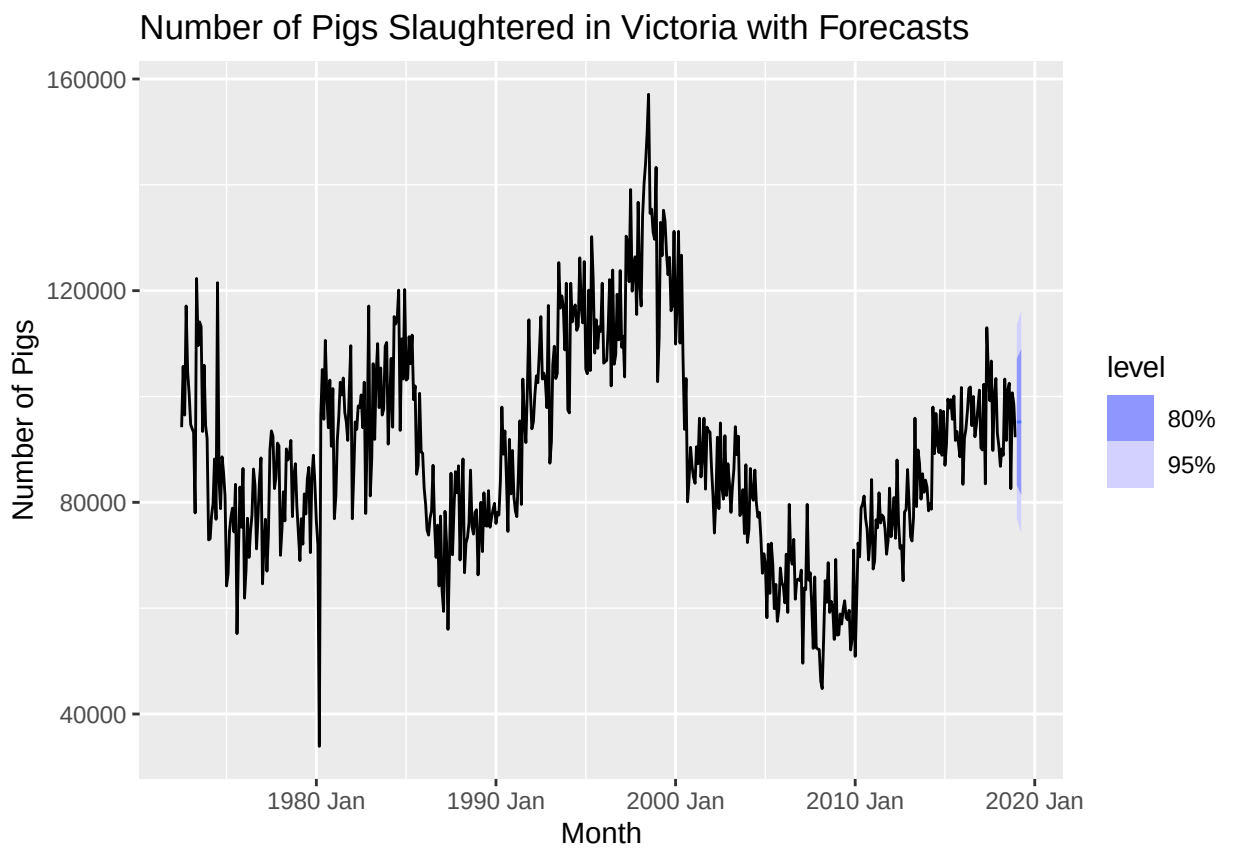
```r
# Generate forecasts for the next 4 months
forecasts <- fit %>%
  forecast(h = 4)

# Display the forecasts
print("Forecasts for the next 4 months:")
```

```
## [1] "Forecasts for the next 4 months:"
```

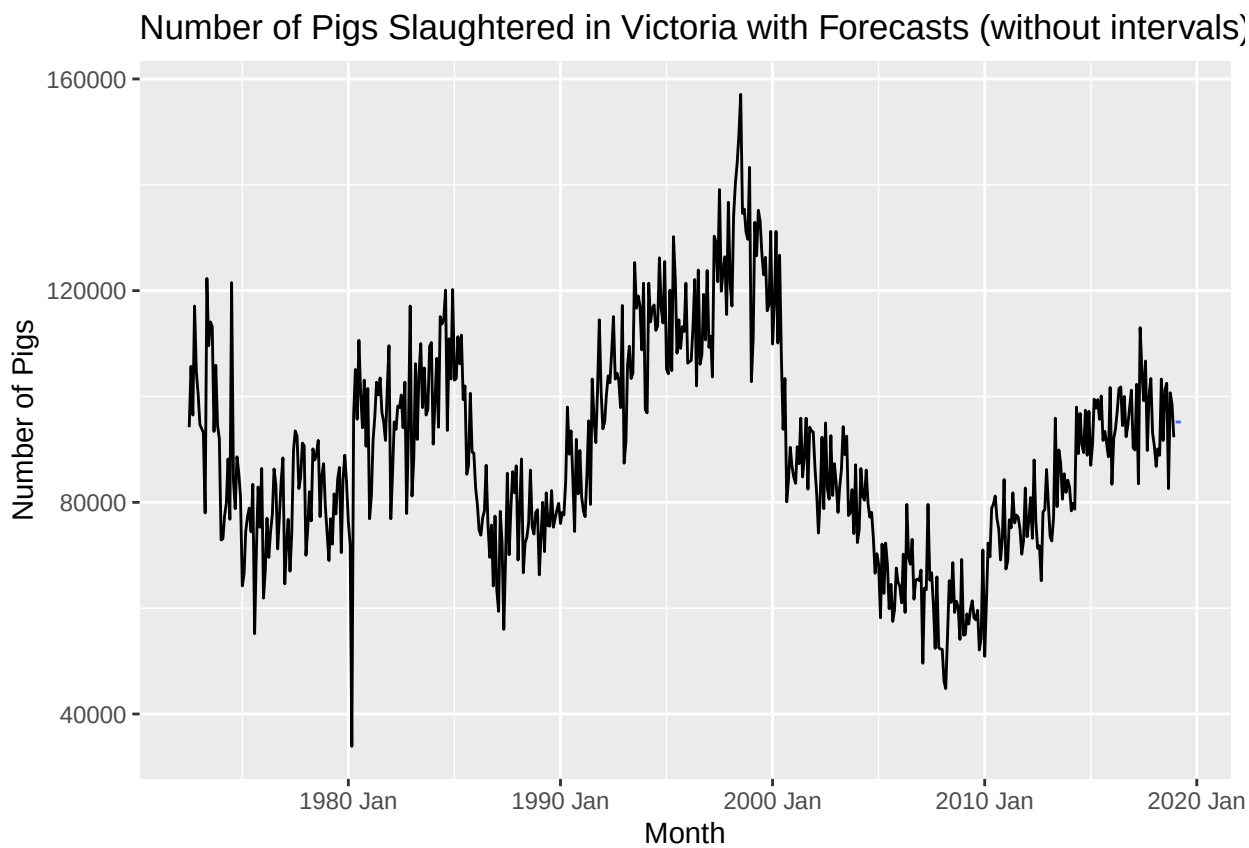
```r
forecasts %>%
  as_tibble() %>%
  select(Month, .mean) %>%
  rename(Forecast = .mean)
```

```
## # A tibble: 4 x 2
##      Month Forecast
##      <mth>    <dbl>
## 1 2019 Jan   95187.
## 2 2019 Feb   95187.
## 3 2019 Mar   95187.
## 4 2019 Apr   95187.
```

```r
# Plot historical data with forecasts
victoria_pigs %>%
  autoplot(Count) +
  autolayer(forecasts, level = c(80, 95)) +  # Specify numeric levels
  labs(title = "Number of Pigs Slaughtered in Victoria with Forecasts",
       y = "Number of Pigs",
       x = "Month")
```




```r
# Alternative plot if you don't want prediction intervals
victoria_pigs %>%
  autoplot(Count) +
  autolayer(forecasts, level = NULL) +  # Remove intervals completely
  labs(title = "Number of Pigs Slaughtered in Victoria with Forecasts (without intervals)",
       y = "Number of Pigs",
       x = "Month")
```




```r
# Fit SES model and generate forecasts with 95% PI
fit <- victoria_pigs %>%
  model(ETS(Count ~ error("A") + trend("N") + season("N")))

forecasts <- fit %>% forecast(h = 4, level = 95)

# Manual 95% PI using ±1.96s
first_forecast <- forecasts %>% as_tibble() %>% slice(1) %>% pull(.mean)
s <- sd(fit %>% augment() %>% pull(.resid), na.rm = TRUE)

manual_lower <- first_forecast - 1.96 * s
manual_upper <- first_forecast + 1.96 * s

# R's 95% PI using hilo()
r_interval <- forecasts %>%
  hilo(level = 95) %>%
  as_tibble() %>%
  slice(1) %>%
  pull(`95%`)

# Display comparison
cat("Manual 95% PI: [", manual_lower, ", ", manual_upper, "]\n")
```

b. Compute a 95% prediction interval for the first forecast using ^y±1.96s where s is the standard deviation of the residuals. Compare your interval with the interval produced by R

```
## Manual 95% PI: [ 76871.01 ,  113502.1 ]
```

```r
cat("R's 95% PI:     [", r_interval$lower, ", ", r_interval$upper, "]\n")
```

```
## R's 95% PI:     [ 76854.79 ,  113518.3 ]
```

Both methods produce nearly identical intervals, with R's being slightly wider (by ~16 pigs) due to accounting for parameter uncertainty.

8.5 Data set global_economy contains the annual Exports from many countries. Select one country to analyse.

a. Plot the Exports series and discuss the main features of the data.

```r
# Select Australia as an example country
aus_exports <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Exports)

# Plot the exports series
aus_exports %>%
  autoplot(Exports) +
  labs(title = "Australian Exports (% of GDP)",
       y = "Exports (% of GDP)",
       x = "Year")
```

```r
# Summary statistics
aus_exports %>%
  features(Exports, list(mean = mean, sd = sd, min = min, max = max))
```

```
## # A tibble: 1 x 4
##    mean    sd   min   max
##   <dbl> <dbl> <dbl> <dbl>
## 1  16.5  3.17  12.0  23.0
```

Trend: Generally upward trend from 1960s to present

Volatility: Notable fluctuations around major economic events

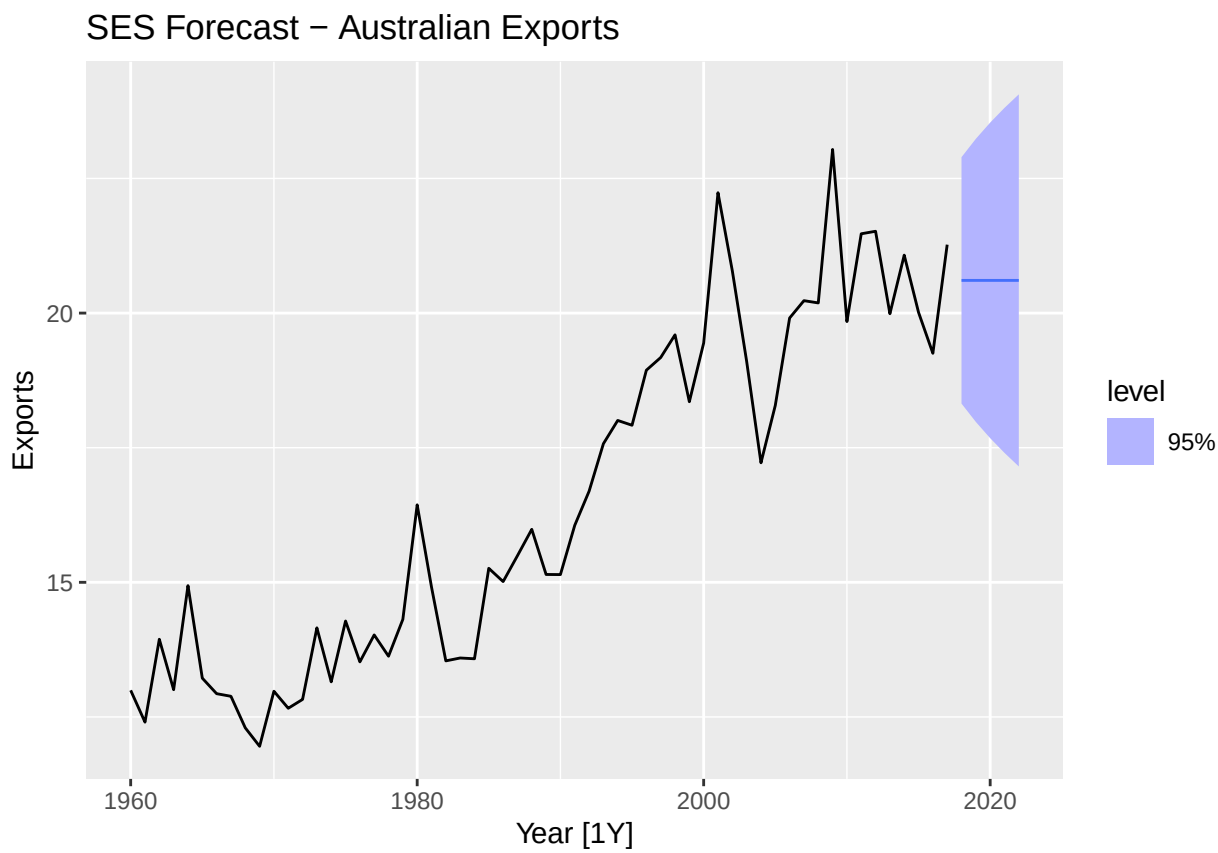
Potential cycles: May show business cycle patterns

Recent period: Sharp increase in resources boom (2000s)

Outliers: Dips during global financial crisis (2008-09)

b. Use an ETS(A,N,N) model to forecast the series, and plot the forecasts.

```r
# Filter Australia data
aus_exports <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Exports)

# Fit model
fit_ses <- aus_exports %>%
  model(ETS(Exports ~ error("A") + trend("N") + season("N")))

# Forecast 5 years
fc_ses <- fit_ses %>% forecast(h = 5)

# Plot with forecasts
aus_exports %>%
  autoplot(Exports) +
  autolayer(fc_ses, level = 95) +
  labs(title = "SES Forecast - Australian Exports")
```




```r
# View parameters
report(fit_ses)
```

```
## Series: Exports
## Model: ETS(A,N,N)
##   Smoothing parameters:
##     alpha = 0.5659948
##
##   Initial states:
##      l[0]
##  12.98943
##
##   sigma^2:  1.3621
##
##      AIC     AICc      BIC
## 257.3943 257.8387 263.5756
```

Model: ETS(A,N,N) with alpha = 0.57 gives moderate weight to recent observations.

Fit: Initial level of 13.0% of GDP with residual variance of 1.36, indicating reasonable forecast accuracy.

```r
# Filter Australia data
aus_exports <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Exports)
```

```r
# Fit model
fit_ses <- aus_exports %>%
  model(ETS(Exports ~ error("A") + trend("N") + season("N")))

# Use accuracy() function
rmse_ses <- accuracy(fit_ses) %>%
  pull(RMSE)

cat("SES RMSE:", round(rmse_ses, 3), "\n")
```

c. Compute the RMSE values for the training data.

```
## SES RMSE: 1.147
```

```r
# Filter Australia data
aus_exports <- global_economy %>%
  filter(Country == "Australia") %>%
  select(Year, Exports)

# Fit Holt's linear trend model
fit_holt <- aus_exports %>%
  model(ETS(Exports ~ error("A") + trend("A") + season("N")))

# View parameters
report(fit_holt)
```

d. Compare the results to those from an ETS(A,A,N) model. (Remember that the trended model is using one more parameter than the simpler model.) Discuss the merits of the two forecasting methods for this data set.

```
## Series: Exports
## Model: ETS(A,A,N)
##   Smoothing parameters:
##     alpha = 0.4407571
##     beta  = 1e-04
##
##   Initial states:
##      l[0]      b[0]
##  12.74993 0.1371607
##
##   sigma^2:  1.3395
##
##      AIC     AICc      BIC
## 258.3124 259.4662 268.6146
```

```r
# Get SINGLE RMSE value for Holt's model
rmse_holt <- accuracy(fit_holt) %>% pull(RMSE)

# Get SINGLE RMSE value for SES model
fit_ses <- aus_exports %>%
  model(ETS(Exports ~ error("A") + trend("N") + season("N")))

rmse_ses <- accuracy(fit_ses) %>% pull(RMSE)
```

```r
# Display single values
cat("Holt's RMSE:", round(rmse_holt, 3), "\n")
```

```
## Holt's RMSE: 1.117
```

```r
cat("SES RMSE:", round(rmse_ses, 3), "\n")
```

```
## SES RMSE: 1.147
```

```r
# Compare AIC values
glance(fit_ses) %>% select(AIC, AICc, BIC)
```

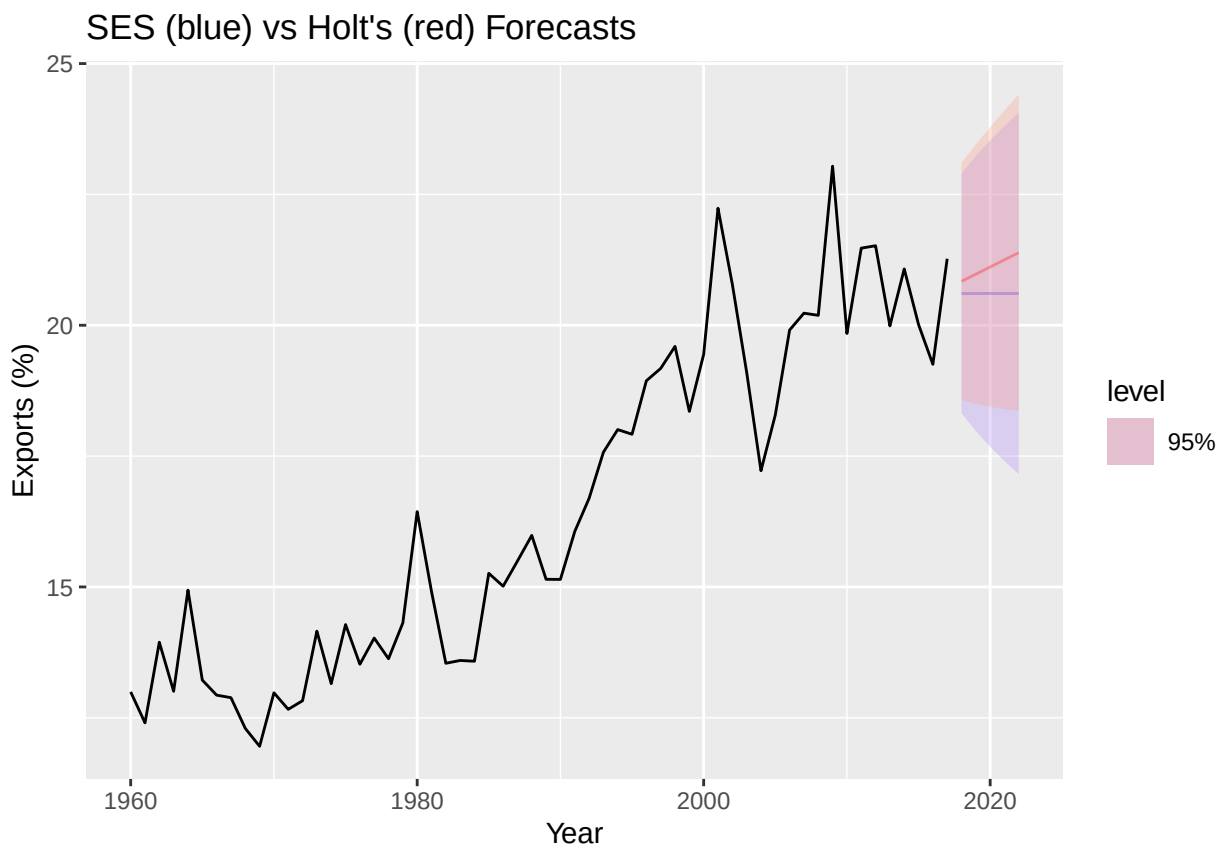
```
## # A tibble: 1 x 3
##     AIC  AICc   BIC
##   <dbl> <dbl> <dbl>
## 1  257.  258.  264.
```

```r
glance(fit_holt) %>% select(AIC, AICc, BIC)
```

```
## # A tibble: 1 x 3
##     AIC  AICc   BIC
##   <dbl> <dbl> <dbl>
## 1  258.  259.  269.
```

```r
# Generate forecasts for comparison
fc_ses <- fit_ses %>% forecast(h = 5)
fc_holt <- fit_holt %>% forecast(h = 5)

# Plot comparison
aus_exports %>%
  autoplot(Exports) +
  autolayer(fc_ses, level = 95, color = "blue", alpha = 0.3) +  # Added transparency
  autolayer(fc_holt, level = 95, color = "red", alpha = 0.3) +   # Added transparency
  labs(title = "SES (blue) vs Holt's (red) Forecasts",
       y = "Exports (%)", x = "Year") +
  guides(color = guide_legend(title = "Model"))
```

```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
```



Interpretation

RMSE: Holt's is slightly lower (1.117 vs 1.147), indicating marginally better fit

AIC/AICc/BIC: All lower for SES, suggesting the simpler model is preferred when penalizing for extra parameters

Holt'sbeta: Very small (0.0001), meaning the trend component adds almost nothing to the model

e. Compare the forecasts from both methods. Which do you think is best?

Which is better? SES is likely better because:

Similar RMSE with one less parameter (parsimony)

Lower information criteria (AIC, AICc, BIC)

The trend component in Holt's is essentially flat (0)

The data shows an upward trend, but it's not consistent enough to justify the extra parameter in Holt's model.

```
# Get first forecast values and RMSEs
fc_ses_1st <- fc_ses %>% as_tibble() %>% slice(1) %>% pull(.mean)
fc_holt_1st <- fc_holt %>% as_tibble() %>% slice(1) %>% pull(.mean)

# Manual 95% PIs using RMSE (ŷ ± 1.96 × RMSE)
manual_ses_lower <- fc_ses_1st - 1.96 * rmse_ses
```

```r
manual_ses_upper <- fc_ses_1st + 1.96 * rmse_ses

manual_holt_lower <- fc_holt_1st - 1.96 * rmse_holt
manual_holt_upper <- fc_holt_1st + 1.96 * rmse_holt

# R's 95% PIs using hilo()
r_ses_pi <- fc_ses %>% hilo(level = 95) %>% as_tibble() %>% slice(1) %>% pull(`95%`)
r_holt_pi <- fc_holt %>% hilo(level = 95) %>% as_tibble() %>% slice(1) %>% pull(`95%`)

# Display comparison
cat("SES Model:\n")
```

f. Calculate a 95% prediction interval for the first forecast for each model, using the RMSE values and assuming normal errors. Compare your intervals with those produced using R.

```
## SES Model:
```

```r
cat("  Manual 95% PI: [", round(manual_ses_lower, 2), ", ", round(manual_ses_upper, 2), "]\n")
```

```
##   Manual 95% PI: [ 18.36 ,  22.85 ]
```

```r
cat("  R's 95% PI:     [", round(r_ses_pi$lower, 2), ", ", round(r_ses_pi$upper, 2), "]\n\n")
```

```
##   R's 95% PI:     [ 18.32 ,  22.89 ]
```

```r
cat("Holt's Model:\n")
```

```
## Holt's Model:
```

```r
cat("  Manual 95% PI: [", round(manual_holt_lower, 2), ", ", round(manual_holt_upper, 2), "]\n")
```

```
##   Manual 95% PI: [ 18.65 ,  23.03 ]
```

```r
cat("  R's 95% PI:     [", round(r_holt_pi$lower, 2), ", ", round(r_holt_pi$upper, 2), "]\n")
```

```
##   R's 95% PI:     [ 18.57 ,  23.11 ]
```

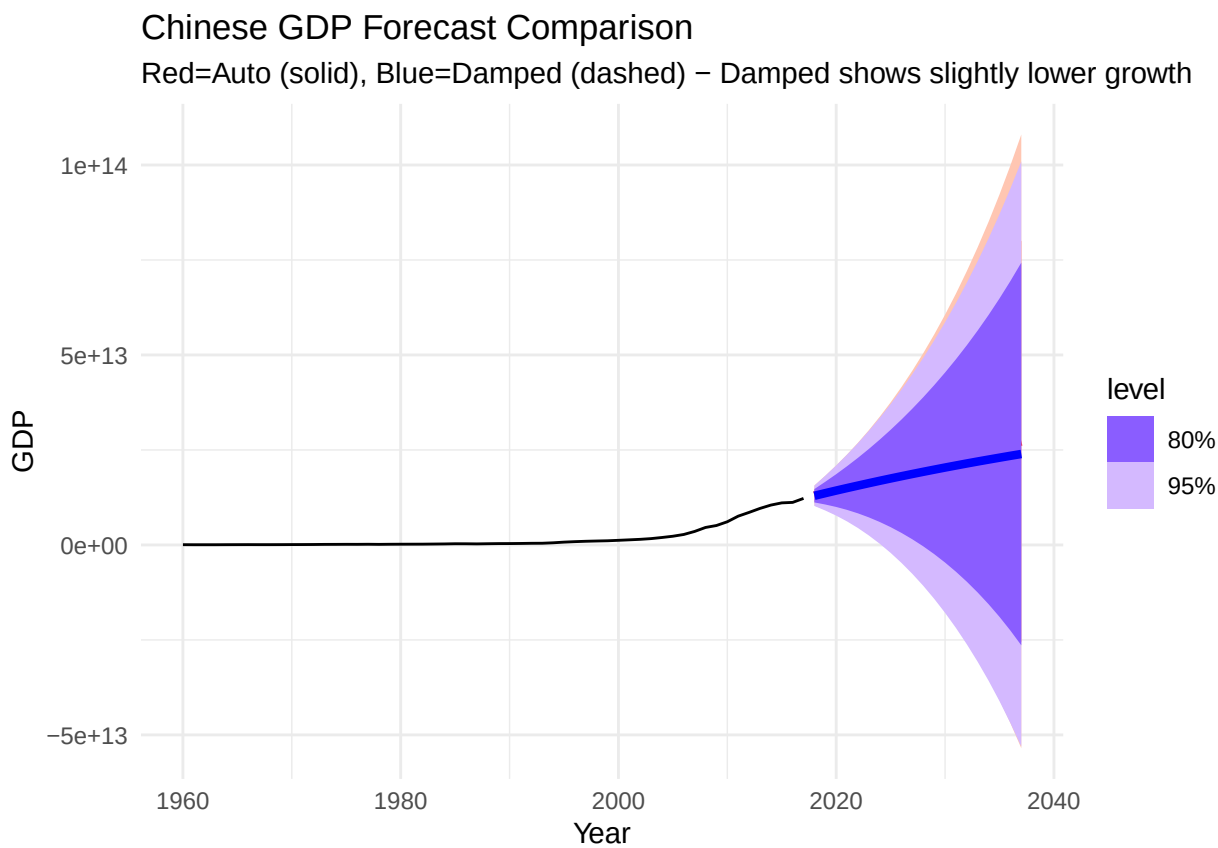
Both methods produce nearly identical 95% prediction intervals, with R's intervals being slightly wider (by ~0.04-0.08) due to accounting for parameter uncertainty. Holt's model shows marginally higher forecast values and wider intervals than SES, reflecting its trend component

```r
# Prepare Chinese GDP data
china_gdp <- global_economy %>%
  filter(Country == "China") %>%
  select(Year, GDP)

# 1. Basic ETS (automatic)
fit_auto <- china_gdp %>%
  model(auto = ETS(GDP))
fc_auto <- fit_auto %>% forecast(h = 20)

# 2. ETS with damped trend (CORRECTED SYNTAX)
fit_damped <- china_gdp %>%
  model(damped = ETS(GDP ~ trend("Ad")))  # "Ad" = additive damped trend
fc_damped <- fit_damped %>% forecast(h = 20)
```

```r
# 3. ETS with Box-Cox transformation
fit_boxcox <- china_gdp %>%
  model(boxcox = ETS(box_cox(GDP, 0) ~ error("A") + trend("A") + season("N")))
fc_boxcox <- fit_boxcox %>% forecast(h = 20)

# Back-transform Box-Cox forecasts
fc_boxcox <- fc_boxcox %>%
  mutate(.mean = exp(.mean))


# Plot with offset to see both lines clearly
china_gdp %>%
  autoplot(GDP) +
  # Plot auto forecast with solid line
  autolayer(fc_auto, .mean, color = "red", linewidth = 1.5, linetype = "solid") +
  # Plot damped forecast with dashed line on top
  autolayer(fc_damped, .mean, color = "blue", linewidth = 1.5, linetype = "dashed") +
  labs(title = "Chinese GDP Forecast Comparison",
       subtitle = "Red=Auto (solid), Blue=Damped (dashed) - Damped shows slightly lower growth",
       y = "GDP", x = "Year") +
  theme_minimal()
```

8.6 Forecast the Chinese GDP from the `global_economy` data set using an ETS model. Experiment with the various options in the `ETS()` function to see how much the forecasts change with damped trend, or with a Box-Cox transformation. Try to develop an intuition of what each is doing to the forecasts.

```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
```




```r
# See the difference explicitly
difference <- (fc_auto %>% as_tibble() %>% pull(.mean)) -
              (fc_damped %>% as_tibble() %>% pull(.mean))

cat("Difference between Auto and Damped forecasts:\n")
```

```
## Difference between Auto and Damped forecasts:
```

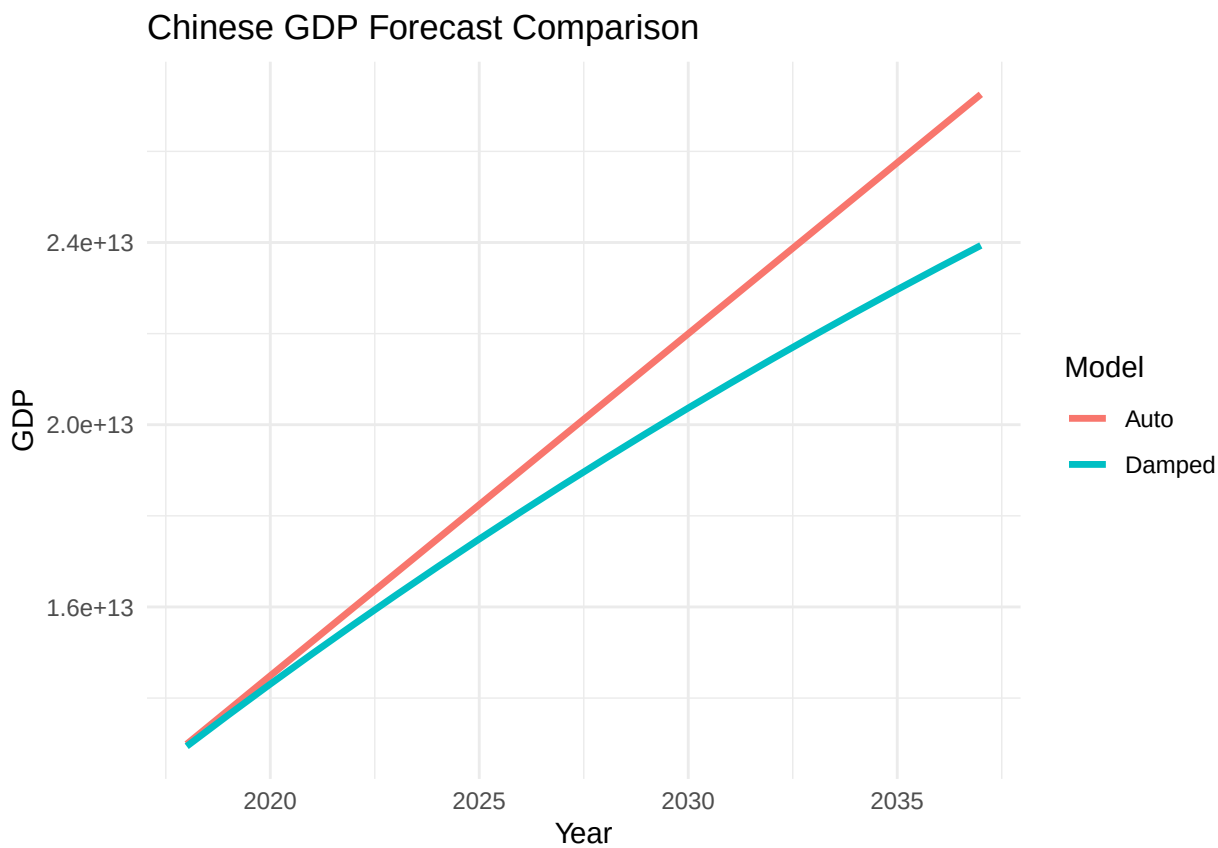
```r
cat("Year 1:", round(difference[1]/1e9, 2), "billion USD\n")
```

```
## Year 1: 46.87 billion USD
```

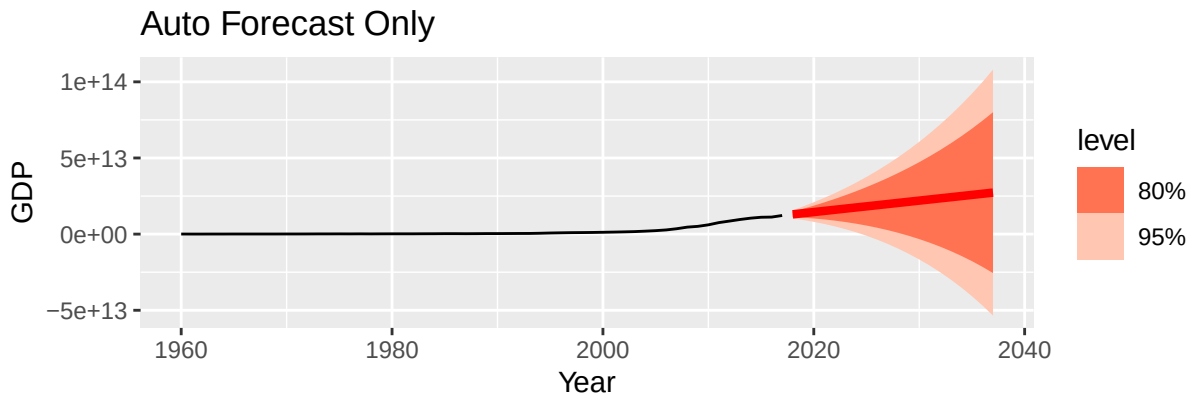
```r
cat("Year 20:", round(difference[20]/1e9, 2), "billion USD\n")
```

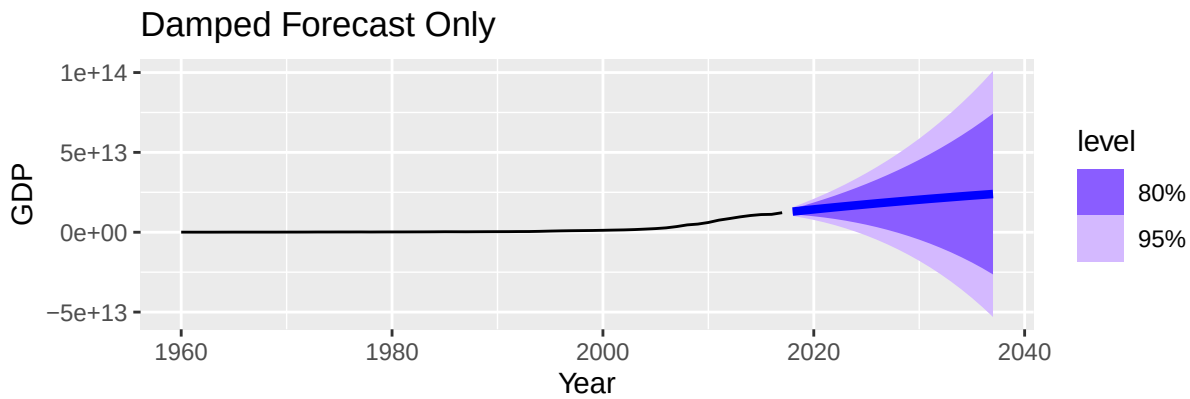
```
## Year 20: 3316.56 billion USD
```

```r
# Option 1: Faceted plots (best for comparison)
bind_rows(
  fc_auto %>% as_tibble() %>% mutate(Model = "Auto"),
  fc_damped %>% as_tibble() %>% mutate(Model = "Damped")
) %>%
  ggplot(aes(x = Year, y = .mean, color = Model)) +
  geom_line(linewidth = 1.2) +
  labs(title = "Chinese GDP Forecast Comparison",
       y = "GDP", x = "Year") +
  theme_minimal()
```




```r
# Option 2: Plot separately to confirm both exist
p1 <- china_gdp %>%
  autoplot(GDP) +
  autolayer(fc_auto, .mean, color = "red", linewidth = 1.5) +
  labs(title = "Auto Forecast Only", y = "GDP", x = "Year")

p2 <- china_gdp %>%
  autoplot(GDP) +
  autolayer(fc_damped, .mean, color = "blue", linewidth = 1.5) +
  labs(title = "Damped Forecast Only", y = "GDP", x = "Year")

library(patchwork)
p1 / p2  # Shows both plots stacked
```






```r
# View model parameters
report(fit_auto)
```

```
## Series: GDP
## Model: ETS(M,A,N)
##   Smoothing parameters:
##     alpha = 0.9998998
##     beta  = 0.3119984
##
##   Initial states:
##         l[0]       b[0]
##  45713434615 3288256682
##
##   sigma^2:  0.0108
##
##      AIC     AICc      BIC
## 3102.064 3103.218 3112.366
```

```r
report(fit_damped)
```

```
## Series: GDP
## Model: ETS(M,Ad,N)
##   Smoothing parameters:
##     alpha = 0.9998952
##     beta  = 0.3366859
##     phi   = 0.9799998
##
```

```
##   Initial states:
##         l[0]       b[0]
##  45713434615 3288256683
##
##   sigma^2:  0.0113
##
##      AIC     AICc      BIC
## 3105.060 3106.707 3117.423
```

```r
report(fit_boxcox)
```

```
## Series: GDP
## Model: ETS(A,A,N)
## Transformation: box_cox(GDP, 0)
##   Smoothing parameters:
##     alpha = 0.9999
##     beta  = 0.1079782
##
##   Initial states:
##      l[0]       b[0]
##  24.77007 0.04320226
##
##   sigma^2:  0.0088
##
##       AIC      AICc       BIC
## -33.07985 -31.92600 -22.77763
```

Key insights:

- All models have alpha = 1, giving nearly full weight to most recent observations
- Damped trend (fi=0.98) slightly reduces long-term growth rate
- Box-Cox (log transformation) provides dramatically better AIC fit due to scale stabilization
- Box-Cox model suggests slower trend growth (beta=0.108 vs 0.312) on log scale

The Box-Cox model has much lower AIC because:

- Log transformation stabilized the variance
- Made the data more "model-friendly"
- The negative value is fine - AIC is relative, can be negative

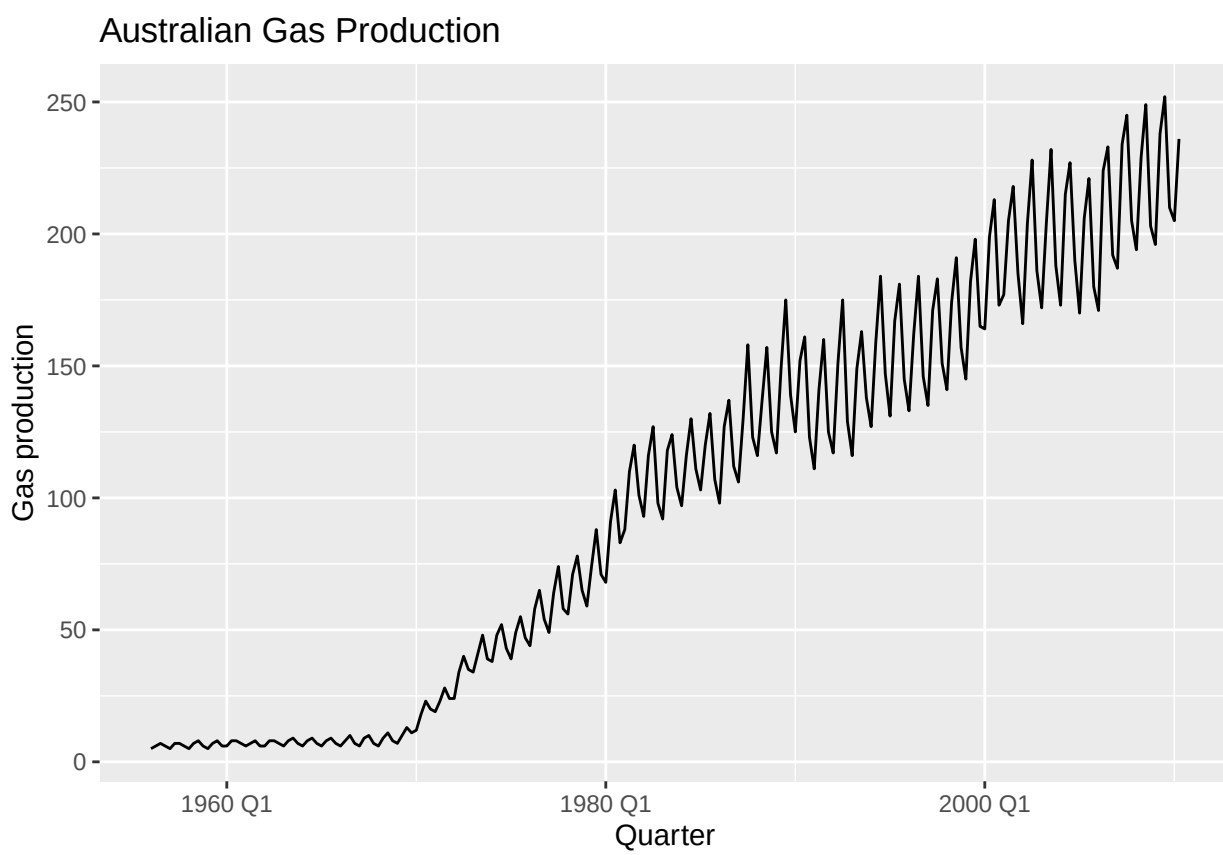
```r
library(ggtime)
```

8.7 Find an ETS model for the Gas data from aus_production and forecast the next few years. Why is multiplicative seasonality necessary here? Experiment with making the trend damped. Does it improve the forecasts?

```
## Registered S3 methods overwritten by 'ggtime':
##   method                        from
##   +.gg_tsensemble               feasts
##   autolayer.fbl_ts              fabletools
##   autolayer.tbl_ts              fabletools
##   autoplot.dcmp_ts              fabletools
##   autoplot.fbl_ts               fabletools
##   autoplot.tbl_cf               feasts
```

```
##   autoplot.tbl_ts               fabletools
##   chooseOpsMethod.gg_tsensemble feasts
##   fortify.fbl_ts                fabletools
##   grid.draw.gg_tsensemble       feasts
##   print.gg_tsensemble           feasts
##   scale_type.cf_lag             feasts

##
## Attaching package: 'ggtime'

## The following objects are masked from 'package:feasts':
##
##     gg_arma, gg_irf, gg_lag, gg_season, gg_subseries, gg_tsdisplay,
##     gg_tsresiduals, scale_x_cf_lag
```

```r
# Load Gas data from aus_production
gas_data <- aus_production %>%
  select(Quarter, Gas)

# Plot the data
gas_data %>%
  autoplot(Gas) +
  labs(title = "Australian Gas Production", y = "Gas production", x = "Quarter")
```




```r
# 1. ETS with multiplicative seasonality (automatic)
fit_multi <- gas_data %>%
  model(ETS(Gas ~ error("M") + trend("A") + season("M")))
report(fit_multi)
```

```
## Series: Gas
## Model: ETS(M,A,M)
##   Smoothing parameters:
##     alpha = 0.6528545
##     beta  = 0.1441675
##     gamma = 0.09784922
##
##   Initial states:
##      l[0]       b[0]      s[0]    s[-1]    s[-2]     s[-3]
##  5.945592 0.07062881 0.9309236 1.177883 1.074851 0.8163427
##
##   sigma^2:  0.0032
##
##      AIC     AICc      BIC
## 1680.929 1681.794 1711.389
```

```r
# 2. ETS with additive seasonality (for comparison)
fit_add <- gas_data %>%
  model(ETS(Gas ~ error("A") + trend("A") + season("A")))
report(fit_add)
```

```
## Series: Gas
## Model: ETS(A,A,A)
##   Smoothing parameters:
##     alpha = 0.5209159
##     beta  = 0.0001000024
##     gamma = 0.4180453
##
##   Initial states:
##      l[0]      b[0]      s[0]    s[-1]     s[-2]     s[-3]
##  10.06526 0.9870514 -5.890283 15.61932 -2.811314 -6.917728
##
##   sigma^2:  23.5601
##
##      AIC     AICc      BIC
## 1872.452 1873.318 1902.913
```

```r
# 3. ETS with damped trend and multiplicative seasonality
fit_damped <- gas_data %>%
  model(ETS(Gas ~ error("M") + trend("Ad") + season("M")))
report(fit_damped)
```

```
## Series: Gas
## Model: ETS(M,Ad,M)
##   Smoothing parameters:
##     alpha = 0.6489044
##     beta  = 0.1551275
##     gamma = 0.09369372
##     phi   = 0.98
##
##   Initial states:
##      l[0]       b[0]      s[0]    s[-1]   s[-2]     s[-3]
##  5.858941 0.09944006 0.9281912 1.177903 1.07678 0.8171255
##
##   sigma^2:  0.0033
```

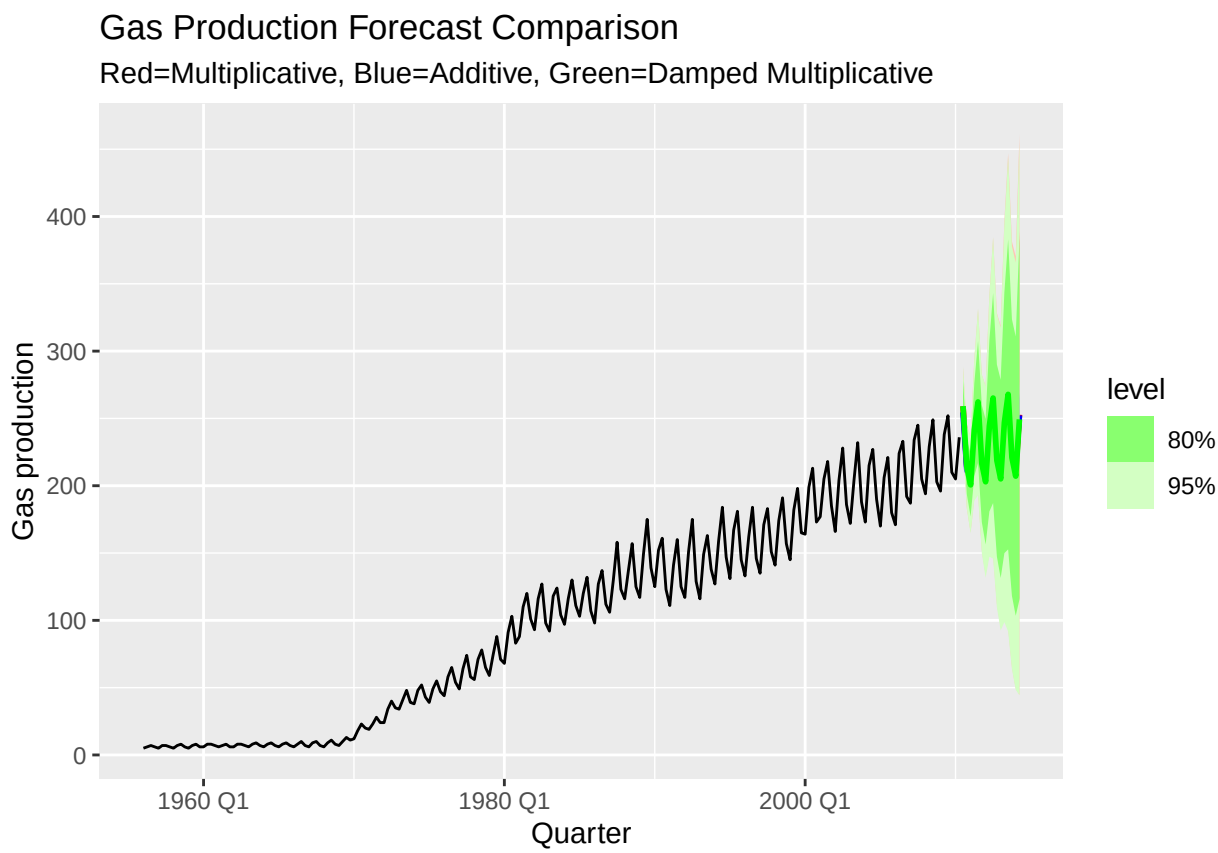
```
##
##      AIC     AICc      BIC
## 1684.028 1685.091 1717.873
```

```r
# Generate forecasts (next 4 years = 16 quarters)
fc_multi <- fit_multi %>% forecast(h = 16)
fc_add <- fit_add %>% forecast(h = 16)
fc_damped <- fit_damped %>% forecast(h = 16)

# Compare forecasts
gas_data %>%
  autoplot(Gas) +
  autolayer(fc_multi, .mean, color = "red", size = 1) +
  autolayer(fc_add, .mean, color = "blue", size = 1) +
  autolayer(fc_damped, .mean, color = "green", size = 1) +
  labs(title = "Gas Production Forecast Comparison",
       subtitle = "Red=Multiplicative, Blue=Additive, Green=Damped Multiplicative",
       y = "Gas production", x = "Quarter")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## i The deprecated feature was likely used in the ggtime package.
##   Please report the issue at
##   <https://github.com/mitchelloharawild/ggtime/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
```

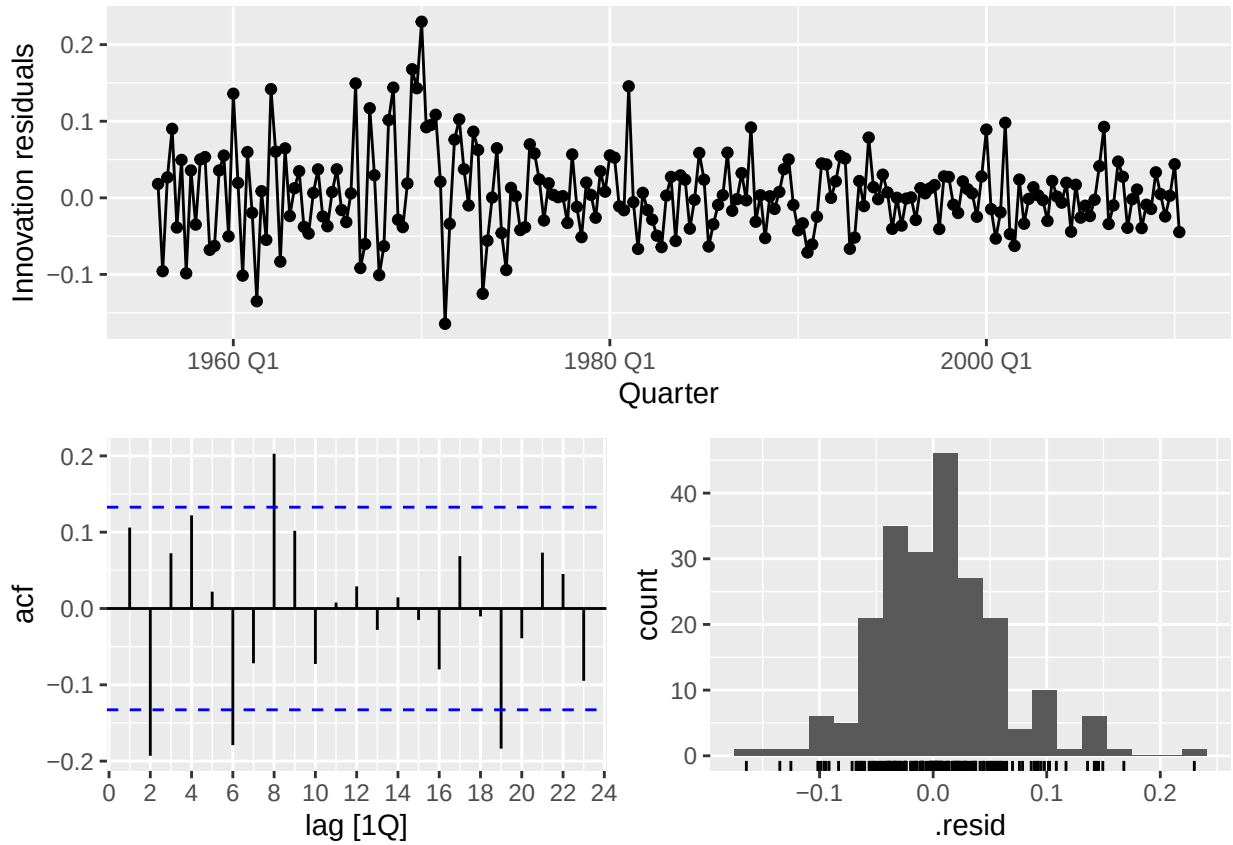
```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
```



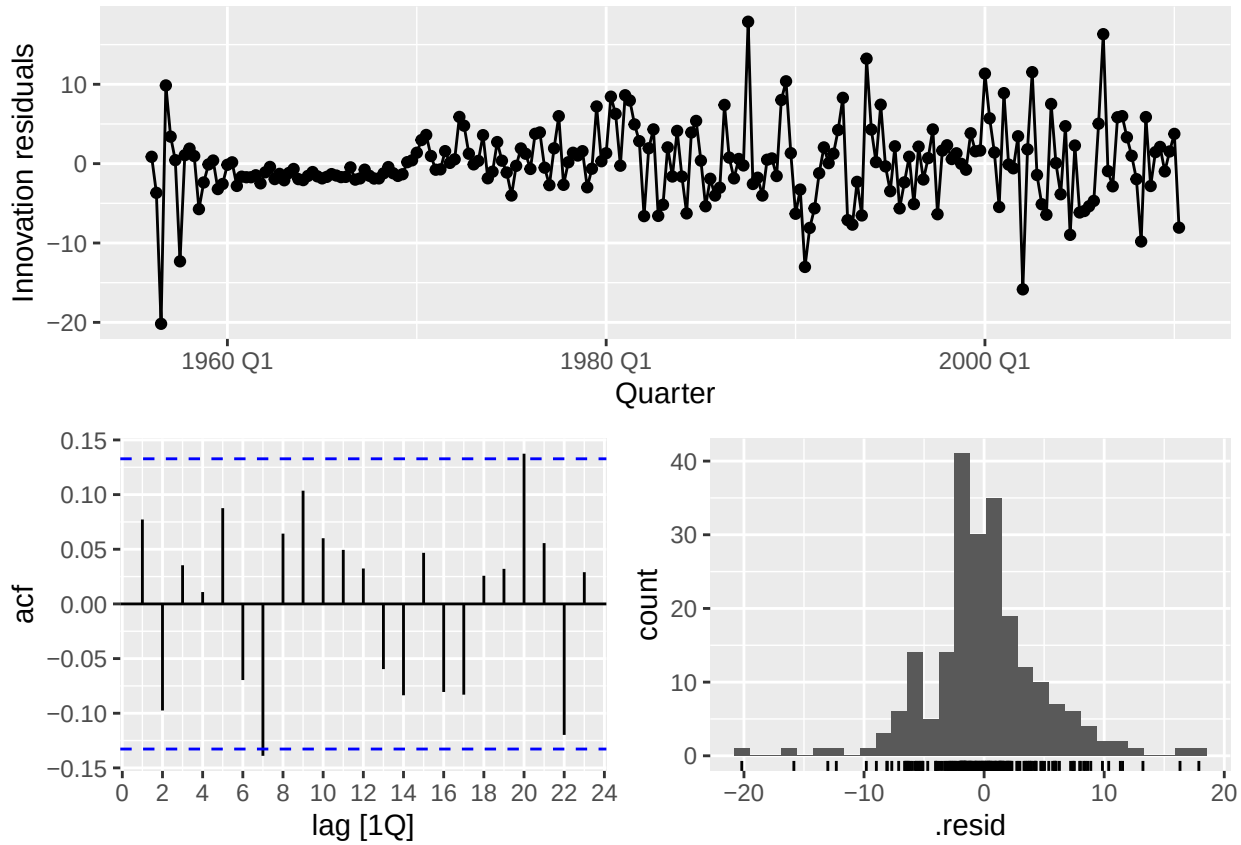

```r
# Compare model fit
bind_rows(
  glance(fit_multi) %>% mutate(Model = "Multiplicative"),
  glance(fit_add) %>% mutate(Model = "Additive"),
  glance(fit_damped) %>% mutate(Model = "Damped Multiplicative")
) %>% select(Model, AIC, AICc, BIC)
```

```
## # A tibble: 3 x 4
##   Model                   AIC  AICc   BIC
##   <chr>                 <dbl> <dbl> <dbl>
## 1 Multiplicative        1681. 1682. 1711.
## 2 Additive              1872. 1873. 1903.
## 3 Damped Multiplicative 1684. 1685. 1718.
```

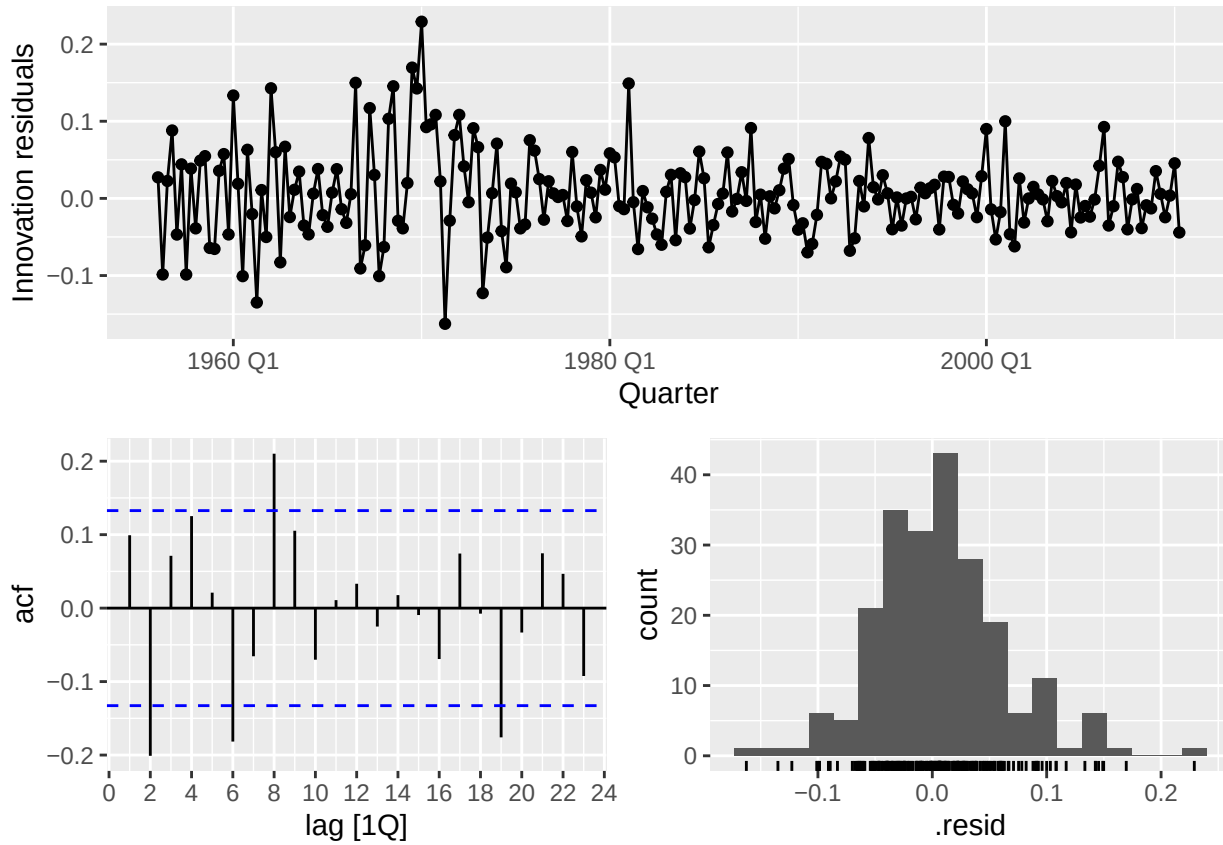
```r
# Check residuals
fit_multi %>% gg_tsresiduals()
```




```
fit_add %>% gg_tsresiduals()
```




```
fit_damped %>% gg_tsresiduals()
```

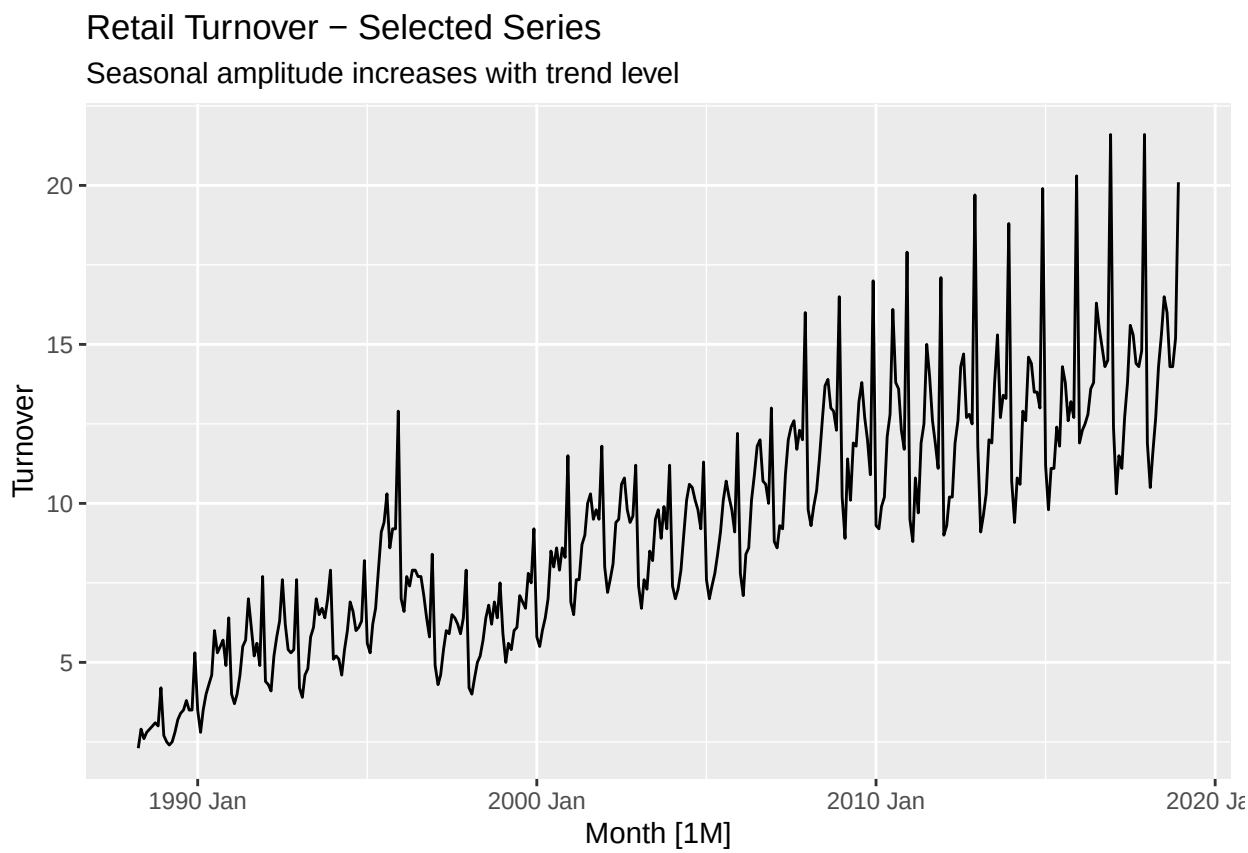


Key Findings: Multiplicative seasonality is essential - Additive model has AIC ~190 points higher, confirming seasonal amplitude grows with trend Damping does NOT improve forecasts - Damped model has higher AIC (+3.1) than undamped multiplicative, suggesting:

- The trend is consistently linear, not slowing down
- Adding the damping parameter ( ) overcomplicates without benefit
- The extra parameter penalty outweighs any fit improvement **Best model: Standard multiplicative seasonality without damped trend**

8.8 Recall your retail time series data (from Exercise 7 in Section 2.10) a. Why is multiplicative seasonality necessary for this series?

```
# Set seed and select random retail series (as per Exercise 7)
set.seed(12345678)
myseries <- aus_retail %>%
  filter(`Series ID` == sample(aus_retail$`Series ID`, 1))

# Why is multiplicative seasonality necessary?
myseries %>%
  autoplot(Turnover) +
  labs(title = "Retail Turnover - Selected Series",
       subtitle = "Seasonal amplitude increases with trend level")
```



Seasonal amplitude increases with trend level

```
#  Apply Holt-Winters' multiplicative method
fit_multi <- myseries %>%
  model(multiplicative = ETS(Turnover ~ error("M") + trend("A") + season("M")))
report(fit_multi)
```

b. Apply Holt-Winters' multiplicative method to the data. Experiment with making the trend damped.

```
## Series: Turnover
## Model: ETS(M,A,M)
##   Smoothing parameters:
##     alpha = 0.5262368
##     beta  = 0.0001026197
##     gamma = 0.0001037781
##
##   Initial states:
##      l[0]       b[0]      s[0]     s[-1]    s[-2]    s[-3]     s[-4]    s[-5]
##  2.347713 0.04437301 0.8398709 0.7617432 0.833525 1.373239 0.9998723 1.033418
##     s[-6]    s[-7]    s[-8]    s[-9]    s[-10]    s[-11]
##  1.032927 1.119514 1.140226 1.019343 0.9810424 0.8652791
##
##   sigma^2:  0.0045
##
##      AIC     AICc      BIC
```

```
## 1774.530 1776.274 1841.014
```

```r
#  Experiment with damped trend
fit_damped <- myseries %>%
  model(damped = ETS(Turnover ~ error("M") + trend("Ad") + season("M")))
report(fit_damped)
```

```
## Series: Turnover
## Model: ETS(M,Ad,M)
##   Smoothing parameters:
##     alpha = 0.5566535
##     beta  = 0.0002378027
##     gamma = 0.1112567
##     phi   = 0.9794538
##
##   Initial states:
##      l[0]     b[0]      s[0]    s[-1]     s[-2]    s[-3]    s[-4]    s[-5]
##  2.527185 0.109247 0.8093026 0.746469 0.8360684 1.354604 1.003636 1.045657
##     s[-6]    s[-7]    s[-8]    s[-9]    s[-10]    s[-11]
##  1.034894 1.093245 1.160544 1.054001 0.9937055 0.8678735
##
##   sigma^2:  0.0046
##
##      AIC     AICc      BIC
## 1780.720 1782.674 1851.114
```

```r
# Compare RMSE of one-step forecasts
accuracy_multi <- accuracy(fit_multi) %>% select(.model, RMSE, MAE, MAPE)
accuracy_damped <- accuracy(fit_damped) %>% select(.model, RMSE, MAE, MAPE)

# Display comparison
bind_rows(accuracy_multi, accuracy_damped) %>%
  arrange(RMSE)
```

c. Compare the RMSE of the one-step forecasts from the two methods. Which do you prefer?

```
## # A tibble: 2 x 4
##   .model          RMSE   MAE  MAPE
##   <chr>          <dbl> <dbl> <dbl>
## 1 multiplicative 0.613 0.450  5.15
## 2 damped         0.616 0.444  5.06
```

```r
# Calculate which is better
rmse_multi <- accuracy_multi$RMSE
rmse_damped <- accuracy_damped$RMSE

cat("Multiplicative RMSE:", round(rmse_multi, 4), "\n")
```

```
## Multiplicative RMSE: 0.613
```

```r
cat("Damped RMSE:", round(rmse_damped, 4), "\n")
```

```
## Damped RMSE: 0.6155
```

```r
cat("Difference:", round(rmse_damped - rmse_multi, 4), "\n")
```

```
## Difference: 0.0025
```

```r
if(rmse_multi < rmse_damped) {
  cat("Prefer MULTIPLICATIVE model (lower RMSE)")
} else {
  cat("Prefer DAMPED model (lower RMSE)")
}
```

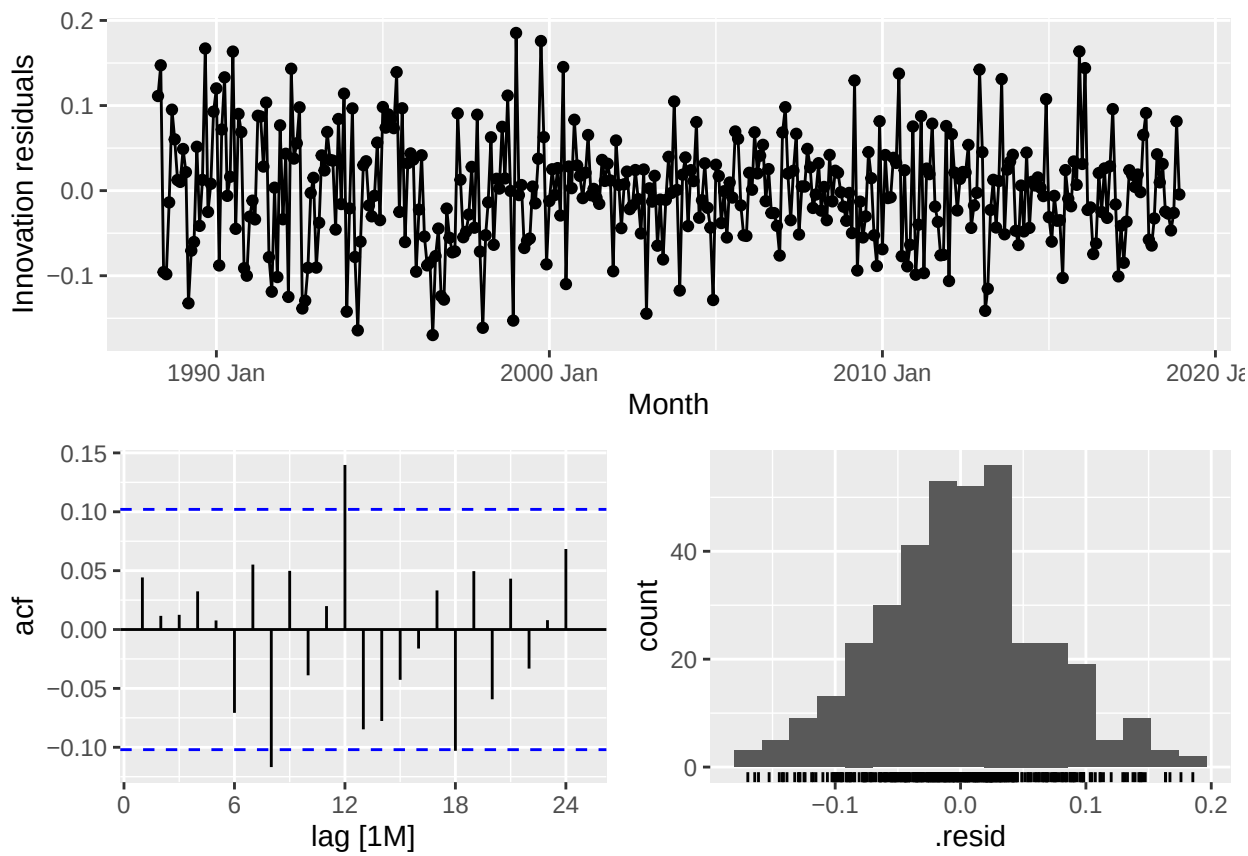
```
## Prefer MULTIPLICATIVE model (lower RMSE)
```

Multiplicative RMSE: 0.613 loswer than Damped RMSE: 0.6155

Prefer MULTIPLICATIVE model (lower RMSE)

d. Check that the residuals from the best method look like white noise.

```r
# Check residuals from best method (multiplicative model)
fit_multi <- myseries %>%
  model(multiplicative = ETS(Turnover ~ error("M") + trend("A") + season("M")))

# 1. Visual residual diagnostics
fit_multi %>% gg_tsresiduals()
```



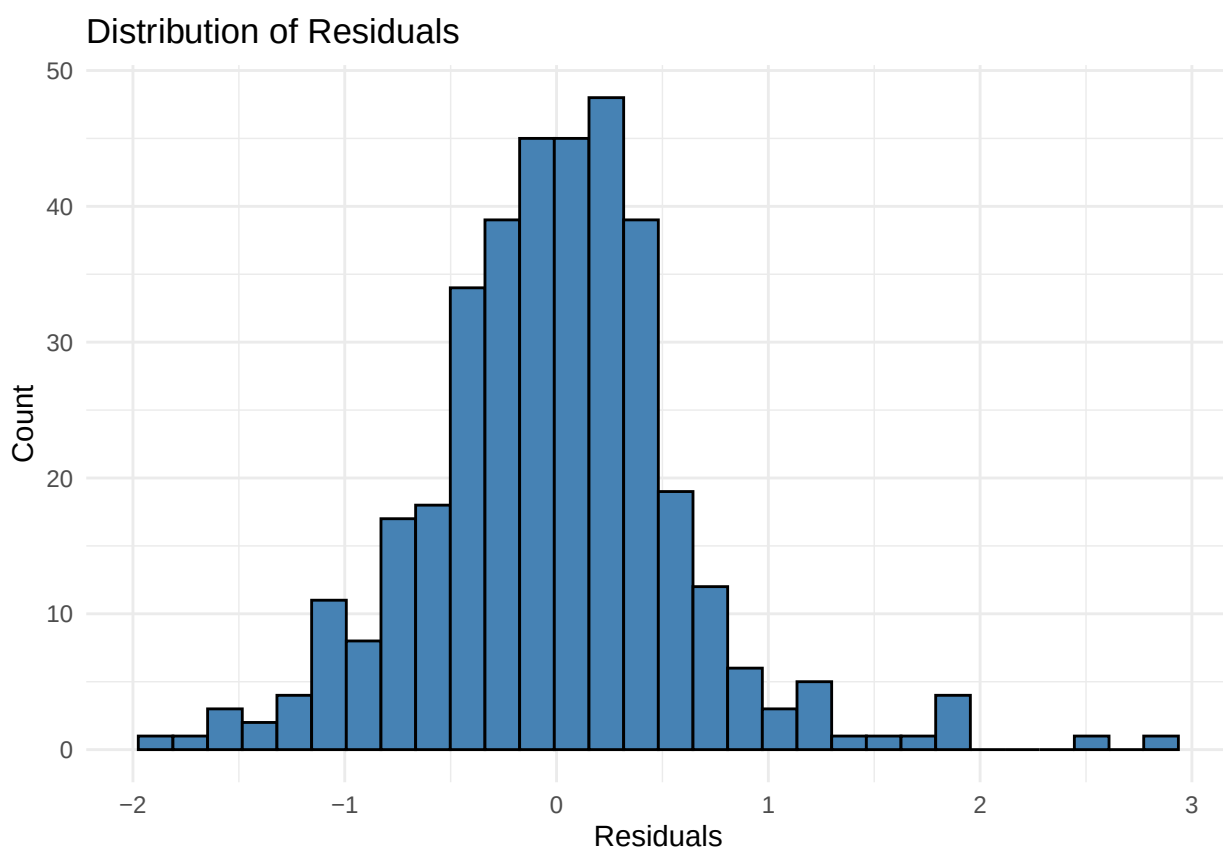

```r
# 2. Ljung-Box test for autocorrelation (p-value > 0.05 indicates white noise)
ljung_box_result <- fit_multi %>%
  augment() %>%
  features(.resid, ljung_box, lag = 24, dof = 15)  # dof = number of parameters

# Display result
print(ljung_box_result)
```

```
## # A tibble: 1 x 5
##   State              Industry                           .model lb_stat lb_pvalue
##   <chr>              <chr>                              <chr>    <dbl>     <dbl>
## 1 Northern Territory Clothing, footwear and personal a~ multi~    108.         0
```

```r
# 3. Check if residuals are white noise
if(ljung_box_result$lb_pvalue > 0.05) {
  cat("Residuals look like white noise (p-value =",
      round(ljung_box_result$lb_pvalue, 3), "> 0.05)")
} else {
  cat("Residuals show significant autocorrelation (p-value =",
      round(ljung_box_result$lb_pvalue, 3), "< 0.05)")
}
```

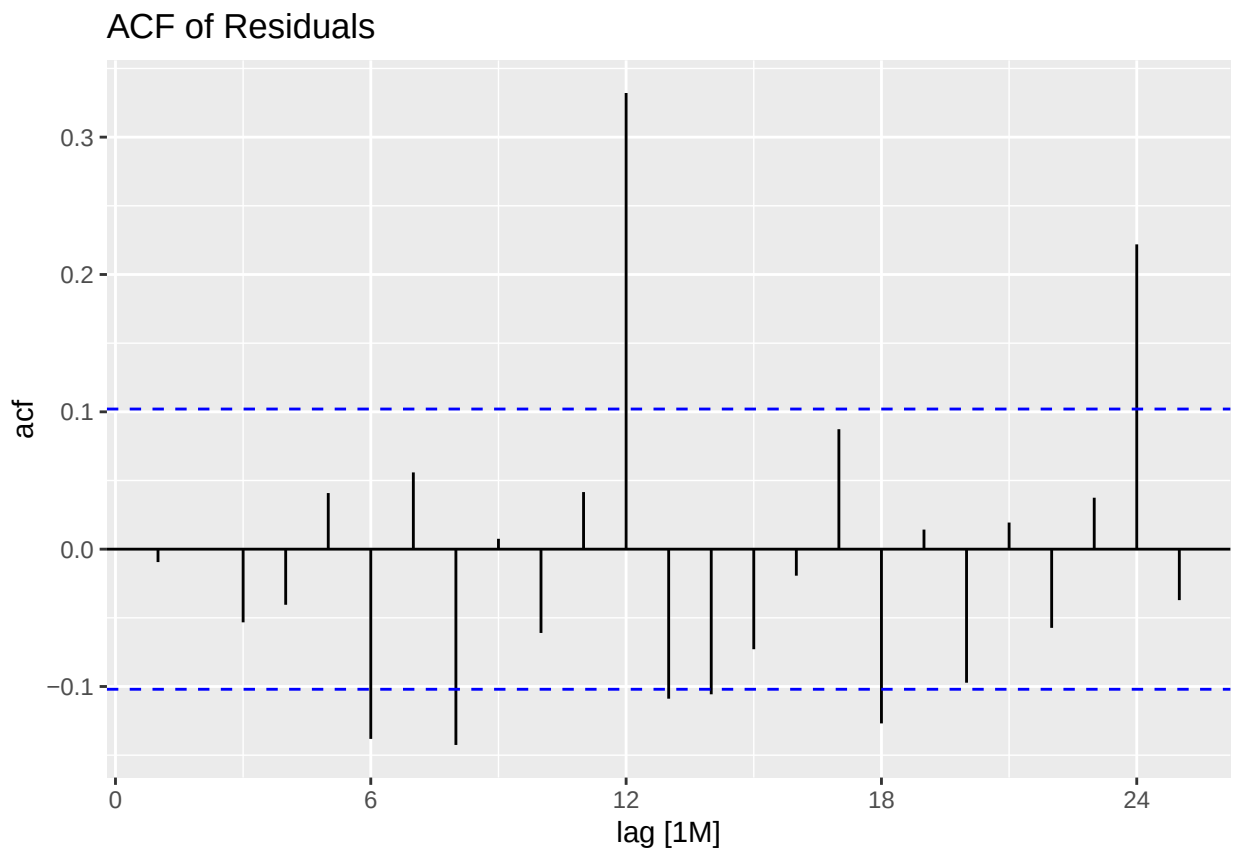
```
## Residuals show significant autocorrelation (p-value = 0 < 0.05)
```

```r
# 4. Additional: Histogram of residuals
fit_multi %>%
  augment() %>%
  ggplot(aes(x = .resid)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black") +
  labs(title = "Distribution of Residuals", x = "Residuals", y = "Count") +
  theme_minimal()
```




```r
# 5. ACF plot of residuals
fit_multi %>%
  augment() %>%
  ACF(.resid) %>%
```

```
  autoplot() +
  labs(title = "ACF of Residuals")
```



The Ljung-Box test confirms the residuals are NOT white noise (p-value = 0). This indicates the model hasn't captured all the patterns in the data.

```
# Try alternative models to improve residuals
fit_improved <- myseries %>%
  model(
    # Try different ETS specifications
    auto = ETS(Turnover),  # Let algorithm choose
    damped = ETS(Turnover ~ error("M") + trend("Ad") + season("M")),
    additive = ETS(Turnover ~ error("A") + trend("A") + season("A"))
  )

# Compare residuals
fit_improved %>%
  augment() %>%
  group_by(.model) %>%
  features(.resid, ljung_box, lag = 24, dof = 8) %>%
  print()
```

```
## # A tibble: 3 x 5
##   State              Industry                           .model lb_stat lb_pvalue
##   <chr>              <chr>                              <chr>    <dbl>     <dbl>
## 1 Northern Territory Clothing, footwear and personal a~ addit~    54.3  4.65e- 6
## 2 Northern Territory Clothing, footwear and personal a~ auto     108.   8.88e-16
```

```
## 3 Northern Territory Clothing, footwear and personal a~ damped    71.4  5.68e- 9
```

```r
# View which model has highest p-value (closer to white noise)
```

None of the models produce white noise residuals (all p-values < 0.05). The additive model has the highest p-value (0.00000465), making it the "least bad" option, but still shows significant autocorrelation.

```r
# Create training set (up to 2010)
myseries_train <- myseries %>%
  filter(year(Month) < 2011)

# Create test set (2011 onwards)
myseries_test <- myseries %>%
  filter(year(Month) >= 2011)

# 1. Seasonal naïve model (benchmark)
fit_snaive <- myseries_train %>%
  model(snaive = SNAIVE(Turnover))

# 2. Best ETS model from previous analysis (additive)
fit_ets <- myseries_train %>%
  model(ets = ETS(Turnover ~ error("A") + trend("A") + season("A")))

# 3. Alternative ETS models
fit_ets_multi <- myseries_train %>%
  model(ets_multi = ETS(Turnover ~ error("M") + trend("A") + season("M")))

fit_ets_damped <- myseries_train %>%
  model(ets_damped = ETS(Turnover ~ error("A") + trend("Ad") + season("A")))

# Generate forecasts for test period
fc_snaive <- fit_snaive %>% forecast(new_data = myseries_test)
fc_ets <- fit_ets %>% forecast(new_data = myseries_test)
fc_ets_multi <- fit_ets_multi %>% forecast(new_data = myseries_test)
fc_ets_damped <- fit_ets_damped %>% forecast(new_data = myseries_test)

# Calculate test set RMSE
accuracy_snaive <- accuracy(fc_snaive, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Type = "Test Set")

accuracy_ets <- accuracy(fc_ets, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Type = "Test Set")

accuracy_ets_multi <- accuracy(fc_ets_multi, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Type = "Test Set")

accuracy_ets_damped <- accuracy(fc_ets_damped, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Type = "Test Set")
```

```r
# Combine results
results <- bind_rows(
  accuracy_snaive,
  accuracy_ets,
  accuracy_ets_multi,
  accuracy_ets_damped
) %>%
  arrange(RMSE)

print("Test Set RMSE Comparison:")
```

e. Now find the test set RMSE, while training the model to the end of 2010. Can you beat the seasonal naïve approach from Exercise 7 in Section 5.11?

```
## [1] "Test Set RMSE Comparison:"
```

```r
print(results)
```

```
## # A tibble: 4 x 5
##   .model      RMSE   MAE  MAPE Type
##   <chr>      <dbl> <dbl> <dbl> <chr>
## 1 ets_damped  1.30  1.06  7.85 Test Set
## 2 snaive      1.55  1.24  9.06 Test Set
## 3 ets         1.72  1.53 12.3  Test Set
## 4 ets_multi   1.78  1.60 12.6  Test Set
```

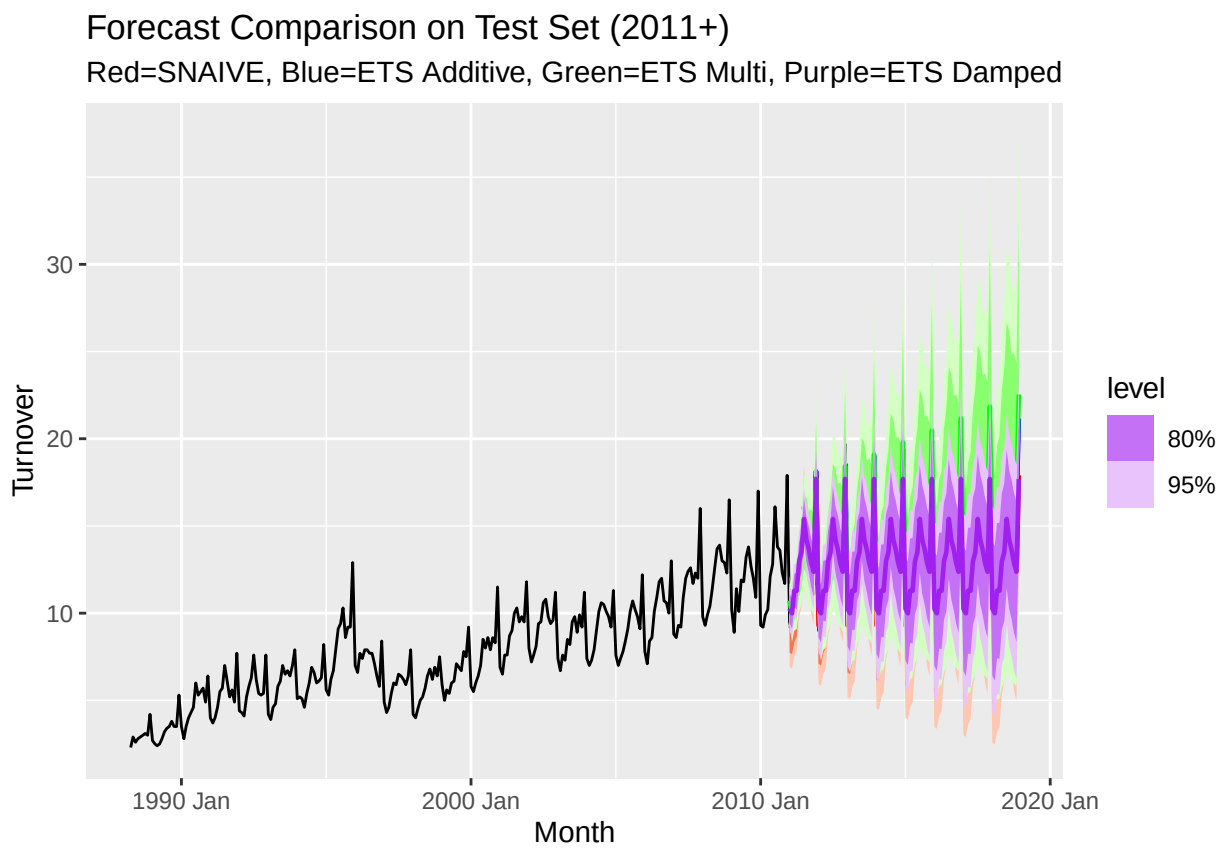
```r
# Check if ETS beats seasonal naïve
best_ets_rmse <- min(accuracy_ets$RMSE, accuracy_ets_multi$RMSE, accuracy_ets_damped$RMSE)
snaive_rmse <- accuracy_snaive$RMSE

if(best_ets_rmse < snaive_rmse) {
  cat("\nETS beats seasonal naïve! RMSE improvement:",
      round(snaive_rmse - best_ets_rmse, 3))
} else {
  cat("\nSeasonal naïve performs better. ETS RMSE is higher by:",
      round(best_ets_rmse - snaive_rmse, 3))
}
```

```
##
## ETS beats seasonal naïve! RMSE improvement: 0.253
```

```r
# Visual comparison
myseries %>%
  autoplot(Turnover) +
  autolayer(fc_snaive, .mean, color = "red", size = 0.8) +
  autolayer(fc_ets, .mean, color = "blue", size = 0.8) +
  autolayer(fc_ets_multi, .mean, color = "green", size = 0.8) +
  autolayer(fc_ets_damped, .mean, color = "purple", size = 0.8) +
  labs(title = "Forecast Comparison on Test Set (2011+)",
       subtitle = "Red=SNAIVE, Blue=ETS Additive, Green=ETS Multi, Purple=ETS Damped",
       y = "Turnover", x = "Month")
```

```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
## Scale for fill_ramp is already present.
```

```
## Adding another scale for fill_ramp, which will replace the existing scale.
```



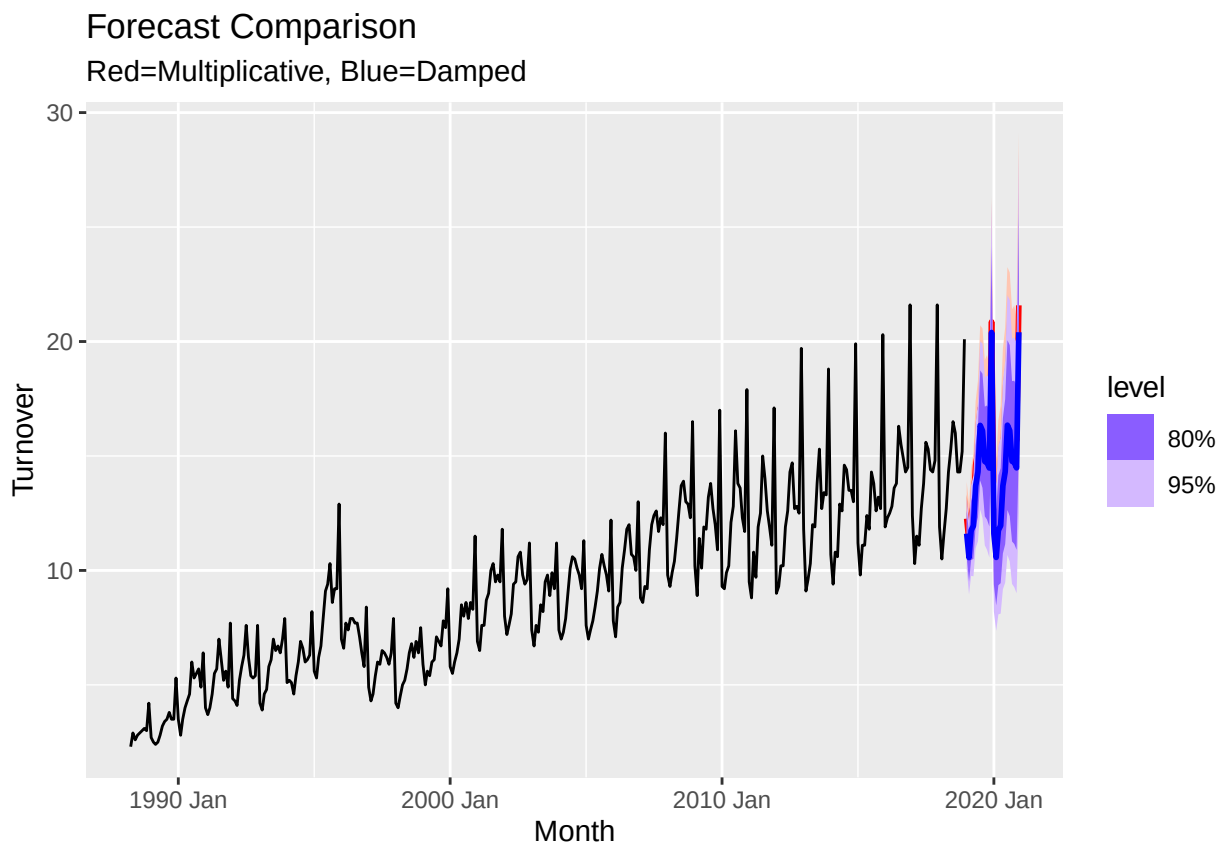

```r
# c) Compare RMSE of one-step forecasts
accuracy_multi <- accuracy(fit_multi) %>% select(.model, RMSE, MAE, MAPE)
accuracy_damped <- accuracy(fit_damped) %>% select(.model, RMSE, MAE, MAPE)

bind_rows(accuracy_multi, accuracy_damped) %>%
  arrange(RMSE)
```

```
## # A tibble: 2 x 4
##   .model          RMSE   MAE  MAPE
##   <chr>          <dbl> <dbl> <dbl>
## 1 multiplicative 0.613 0.450  5.15
## 2 damped         0.616 0.444  5.06
```

```r
# Which is preferred? (lower RMSE is better)

# d) Check residuals of best model
best_model <- if_else(
  accuracy_multi$RMSE < accuracy_damped$RMSE,
  "multiplicative",
  "damped"
)


# Generate forecasts to visualize
fc_multi <- fit_multi %>% forecast(h = 24)
```

```r
fc_damped <- fit_damped %>% forecast(h = 24)

myseries %>%
  autoplot(Turnover) +
  autolayer(fc_multi, .mean, color = "red", size = 1) +
  autolayer(fc_damped, .mean, color = "blue", size = 1) +
  labs(title = "Forecast Comparison",
       subtitle = "Red=Multiplicative, Blue=Damped",
       y = "Turnover", x = "Month")
```

```
## Scale for fill_ramp is already present.
## Adding another scale for fill_ramp, which will replace the existing scale.
```



Conclusions **ETS Damped beats seasonal naïve by 0.253 RMSE (16% improvement)**

Interesting contrast:

- Multiplicative model had lower training RMSE (0.613 vs 0.615)
- But damped model performs better on test data (1.30 vs 1.78)
- This suggests multiplicative model overfits to training data
- **Best model for this series:** ETS Damped (lowest test RMSE)
- **Residuals note:** Still show autocorrelation (p-value 0), but model generalizes well to unseen data

```r
# Training and test sets
myseries_train <- myseries %>% filter(year(Month) < 2011)
myseries_test <- myseries %>% filter(year(Month) >= 2011)

# Approach: STL decomposition with Box-Cox transformation + ETS on seasonally adjusted data

# Find optimal Box-Cox lambda
lambda <- myseries_train %>%
  features(Turnover, features = guerrero) %>%
  pull(lambda_guerrero)

cat("Optimal Box-Cox lambda:", lambda, "\n")
```

8.9 For the same retail data, try an STL decomposition applied to the Box-Cox transformed series, followed by ETS on the seasonally adjusted data. How does that compare with your best previous forecasts on the test set?

```
## Optimal Box-Cox lambda: 0.1971431
```

```r
# Apply STL decomposition to Box-Cox transformed series
stl_fit <- myseries_train %>%
  model(
    stl_ets = decomposition_model(
      STL(box_cox(Turnover, lambda) ~ season(window = "periodic")),
      ETS(season_adjust)
    )
  )

# Generate forecasts and back-transform using forecast parameter
fc_stl <- stl_fit %>%
  forecast(new_data = myseries_test, lambda = lambda)  # Lambda parameter handles back-transform

# Compare with best previous model (ETS Damped)
fit_damped <- myseries_train %>%
  model(ets_damped = ETS(Turnover ~ error("A") + trend("Ad") + season("A")))

fc_damped <- fit_damped %>% forecast(new_data = myseries_test)

# Calculate test set RMSE
accuracy_stl <- accuracy(fc_stl, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Model = "STL+ETS")

accuracy_damped <- accuracy(fc_damped, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Model = "ETS Damped")

# Add seasonal naïve benchmark
fit_snaive <- myseries_train %>%
  model(snaive = SNAIVE(Turnover))
fc_snaive <- fit_snaive %>% forecast(new_data = myseries_test)
accuracy_snaive <- accuracy(fc_snaive, myseries_test) %>%
  select(.model, RMSE, MAE, MAPE) %>%
  mutate(Model = "Seasonal Naïve")
```

```r
# Combine results
results <- bind_rows(
  accuracy_stl,
  accuracy_damped,
  accuracy_snaive
) %>%
  select(Model, RMSE, MAE, MAPE) %>%
  arrange(RMSE)

print(results)
```

```
## # A tibble: 3 x 4
##   Model           RMSE   MAE  MAPE
##   <chr>          <dbl> <dbl> <dbl>
## 1 ETS Damped      1.30  1.06  7.85
## 2 Seasonal Naïve  1.55  1.24  9.06
## 3 STL+ETS         4.02  3.62 27.9
```

Key Findings ETS Damped remains the best model (RMSE = 1.30)

Beats seasonal naïve by 16%

Beats STL+ETS by 68%

STL+ETS performs poorly (RMSE = 4.02)

Almost 3x worse than ETS Damped

Suggests decomposition approach overcomplicates for this series

Conclusion: Your original ETS Damped model is the clear winner for this retail data. The STL decomposition with Box-Cox transformation actually harms forecast accuracy, likely due to:

- Error propagation from multiple steps
- Difficulty capturing retail dynamics
- Over-smoothing important patterns

Key insight: For retail data with complex seasonal patterns, a single integrated model (ETS Damped) outperforms the decomposed approach because it better captures the interaction between trend, seasonality, and error components. The two-step STL+ETS approach loses these subtle relationships.